

Refresh Token과 Redis 기반 토큰 저장소

학습 체크리스트 (1/2)

☐ 무상태 JWT가 로그아웃·탈취 상황에서 갖는 한계를 설명할 수 있다.

☐ Access Token과 Refresh Token의 역할을 분리하여 이해할 수 있다.

☐ 클라우드 Redis(Valkey) 인스턴스를 준비할 수 있다.

☐ `.env` 와 `spring.config.import` 로 외부 설정을 구성할 수 있다.

학습 체크리스트 (2/2)

☐

@RedisHash 기반 토큰 엔티티를 설계할 수 있다.

☐

jti · type 클레임을 포함한 토큰을 발급할 수 있다.

☐

HttpOnly 쿠키 속성을 상황에 맞게 결정할 수 있다.

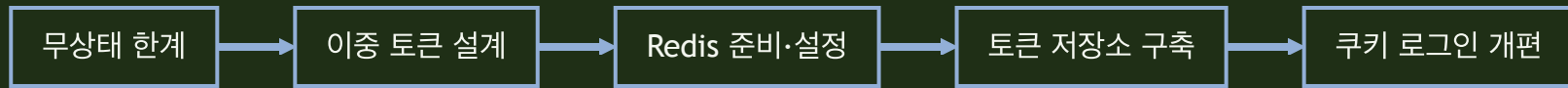
☐

로그인 API 응답 구조를 Access/Refresh로 개편할 수 있다.

학습 범위와 선수 조건

- 선수 지식: `JwtTokenProvider` · `JwtAuthenticationFilter` 기반 무상태 인증 구성
- 핵심 질문: 짧은 토큰 수명과 지속적인 로그인 유지를 어떻게 동시에 만족시킬까
- 기준 환경: Spring Boot 4 · Spring Security 7 계열
- 다루지 않는 범위: 무음 재발급·재사용 감지·블랙리스트 처리는 다음 차시

전체 진행 순서



무상태 JWT의 한계

만료된 Access Token이 만드는 재로그인

- 짧은 수명의 대가: Access Token 유효기간(예: 30분)이 끝나면 인증이 즉시 실패
- 반복되는 재입력: 만료될 때마다 아이디·비밀번호를 다시 입력해야 함
- 사용성 저하: 작업 중간에 세션이 끊겨 흐름이 자주 끊김
- 딜레마: 수명을 늘리면 편해지지만 탈취 시 피해 노출 시간이 늘어남

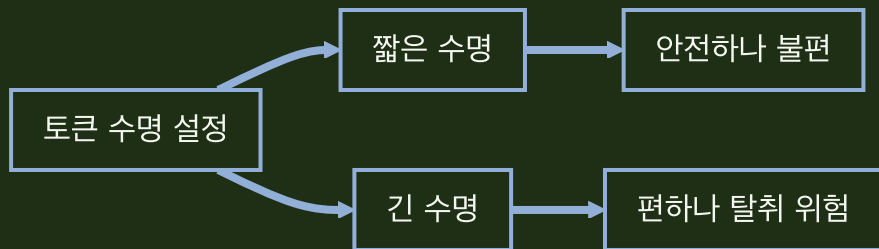
서버가 토큰에 대해 알 수 없는 것

- 검증 범위: 서버는 서명 검증과 만료 시간(`exp`)만으로 유효성을 판단
- 알 수 없는 현재 상태: 발급 이후 로그아웃했는지, 계정이 정지됐는지는 토큰만으로 알 수 없음
- 저장하지 않는 구조: 무상태 특성상 서버가 발급 이력을 따로 기억하지 않음
- 결과: 클라이언트가 토큰을 지워도 만료 전까지 서버는 계속 유효하다고 판단

세션 방식과 무상태 JWT의 차이

상황	세션 방식	무상태 JWT
로그아웃	저장소 삭제로 즉시 무효화	만료 전까지 서버는 유효로 판단
토큰 탈취	세션 삭제로 즉시 차단	만료 시간까지 공격자 호출 가능
계정 차단	해당 세션만 삭제하면 즉시 차단	이미 발급된 토큰은 만료 전까지 통과

수명 설정의 딜레마



핵심 용어 정리

무상태 (Stateless)

서버가 클라이언트의 이전 요청 상태를 저장하지 않고 매 요청을 독립적으로 처리하는 방식

토큰 무효화 (Token Revocation)

아직 만료되지 않은 토큰을 서버가 의도적으로 더 이상 유효하지 않게 만드는 것

Access Token과 Refresh Token

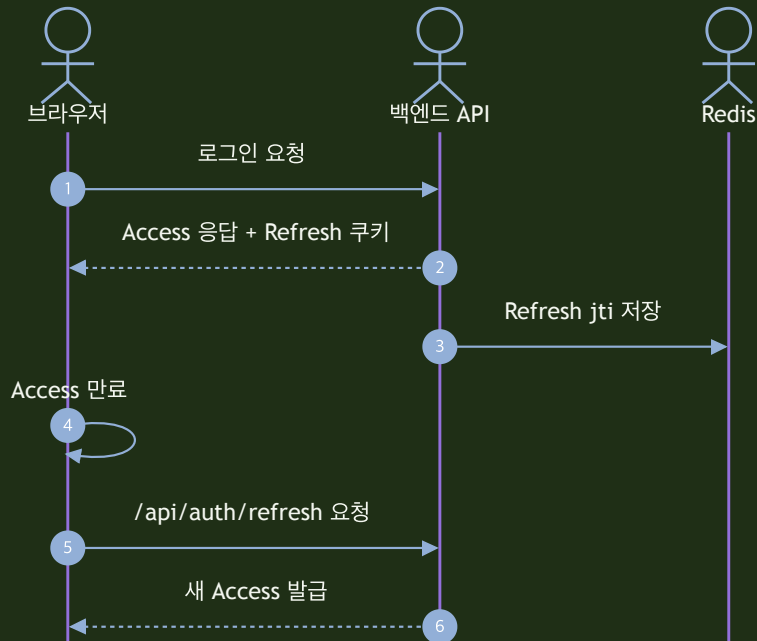
하나의 토큰에 모든 책임을 지우지 않기

- 문제의 근원: 인가와 재발급이라는 두 역할을 토큰 하나가 모두 담당
- 분리 아이디어: 인가 전용 토큰과 재발급 전용 토큰으로 역할을 나눔
- 인가 전용: 매 요청 검증만 담당, 수명을 짧게 유지해 피해 범위 축소
- 재발급 전용: 새 인가 토큰 발급만 담당, 서버가 발급 이력을 직접 관리

두 토큰의 역할 비교

항목	Access Token	Refresh Token
수명	짧음 (예: 15분)	깊 (예: 14일)
전송 경로	매 요청 <code>Authorization</code> 헤더	<code>/api/auth/refresh</code> 시 쿠키 자동 전송
탈취 시 피해	최대 15분간 호출 가능	Redis 삭제 시 즉시 재발급 차단

로그인부터 재발급까지의 흐름



서버가 Refresh Token만 기억하는 이유

- 차단 가능성 확보: 발급 이력을 남겨야 필요할 때 즉시 재발급을 막을 수 있음
- Access까지 저장하면: 매 요청마다 저장소 조회가 필요해 무상태의 성능 이점 상실
- 역할 분담: Access는 빠른 검증, Refresh는 신뢰할 수 있는 이력 관리
- 결과: 최소한의 저장으로 보안과 성능을 함께 확보

핵심 용어 정리

JWT 고유 식별자 (jti)

토큰마다 부여되는 고유 UUID로, Redis에 저장할 때 세션 키 역할을 함

무음 재발급 (Silent Refresh)

사용자가 인지하지 못하는 사이 Refresh Token으로 새 Access Token을 자동 발급받는 방식

Redis 저장소 준비와 외부 설정

토큰 이력을 어디에 남길 것인가

- 관계형 DB의 한계: 매 재발급 요청마다 디스크 기반 조회가 발생해 지연 시간 증가
- 인메모리 저장소: 메모리 기반 조회로 낮은 지연 시간 확보, 세션성 데이터에 적합
- 만료 데이터 정리: 유효 기간이 지난 발급 이력을 자동으로 삭제해야 별도 배치 불필요
- Redis 채택 이유: 빠른 응답 속도와 TTL 자동 만료 기능을 동시에 제공

관계형 데이터베이스와 Redis의 역할 차이

비교 항목	관계형 DB	Redis
저장 위치	디스크 기반	메모리 기반
조회 속도	상대적으로 느림	매우 빠름
만료 처리	별도 배치·스케줄러 필요	TTL로 자동 만료
적합한 데이터	영구 보관 데이터	세션·이력 등 휘발성 데이터

Valkey와 클라우드 매니지드 서비스

- **Valkey**: Redis의 오픈소스 포크로 Redis 프로토콜 및 명령어 호환
- **매니지드 서비스**: 로컬 설치 없이 클라우드에서 무료로 바로 사용 가능
- **Aiven for Valkey**: 무료 플랜을 제공하는 추천 매니지드 서비스
- **Upstash Redis**: 서버리스 방식의 대안 매니지드 서비스

Aiven for Valkey 인스턴스 생성 절차

단계	조작 과정	확인 포인트
1	console.aiven.io 가입	계정 생성 및 로그인 완료
2	Create service에서 Valkey Free 플랜 선택	요금제가 Free인지 확인
3	서비스 생성 완료 대기	상태가 Running으로 전환
4	서비스 개요에서 Service URI 복사	<code>rediss://</code> 스키마 포함 여부

TLS 연결 문자열의 구조

```
1 rediss://default:AVNS_XXXXXX@valkey-XXXX.aivencloud.com:12345
```

- `rediss://` 는 TLS 암호화가 적용된 스킴으로, 일반 `redis://` 와는 구분됨

민감 정보를 코드에서 분리하기

- **노출 금지:** Redis 접속 URI와 JWT 시크릿 키를 코드나 Git 저장소에 직접 기록 금지
- **외부화:** `.env` 파일에 민감 정보를 선언해 소스 코드와 분리
- **자동 로드:** `spring.config.import` 로 `.env` 를 설정에 자동 바인딩
- **버전 관리 제외:** `.gitignore` 에 `.env` 를 등록해 커밋 방지

.env 파일에 접속 정보 선언

```
1  JWT_SECRET=dGhpcy1pcy1hLXNhbXBsZS1zZWNyZXQ ...  
2  REDIS_URL=rediss://default:AVNS_xxx@valkey-xxxx.aivencloud.com:12345
```

- 두 값 모두 소스 코드에는 등장하지 않고 `.env` 파일에서만 관리됨

application.yml의 Redis 연결 설정

```
1  spring:
2    config:
3      import: optional:file:.env[.properties]
4    data:
5      redis:
6        url: ${REDIS_URL}
7        timeout: 2000ms
```

- `optional:` 접두어로 `.env` 파일이 없어도 애플리케이션 구동에 지장 없음

application.yml의 토큰·쿠키 설정

```
1  app:
2    jwt:
3      secret: ${JWT_SECRET}
4      access-token-validity: 15m
5      refresh-token-validity: 14d
6    auth:
7      refresh-cookie-name: refresh_token
8      cookie-secure: false
```

- 토큰 유효 기간과 쿠키 이름·보안 옵션을 코드 밖에서 관리

설정 프로퍼티 확인 사항

프로퍼티	지정 값	유의점
<code>spring.config.import</code>	<code>optional:file:.env[.properties]</code>	<code>optional:</code> 없으면 파일 부재 시 구동 실패
<code>spring.data.redis.url</code>	<code>\${REDIS_URL}</code>	Spring Boot 3.x 이후 <code>spring.redis.*</code> 에서 경로 변경
<code>app.auth.cookie-secure</code>	<code>false</code> / <code>true</code>	로컬 HTTP는 false, 운영 HTTPS는 true

Spring Data Redis 의존성 추가

```
1 dependencies {  
2     implementation 'org.springframework.boot:spring-boot-starter-data-redis'  
3 }
```

- 이 스타터 하나로 Redis 연결과 `@RedisHash` 기반 Repository 사용이 가능해짐

핵심 용어 정리

유효 시간 (TTL, Time To Live)

저장된 데이터가 자동으로 삭제되기까지 남은 시간을 지정하는 값

매니지드 서비스 (Managed Service)

설치·운영·백업 등을 클라우드 제공자가 대신 관리해주는 서비스 형태

Redis 토큰 저장소 구축

RedisTemplate 없이 객체 단위로 다루기

- 선언적 저장소: `@RedisHash` 와 `CrudRepository` 로 JPA와 비슷한 사용 감각 제공
- 자동 만료 지원: `@TimeToLive` 필드만 두면 별도 배치 없이 TTL 만료 처리
- 직렬화 설정 불요: `RedisTemplate` 직접 구성 없이 객체 단위 CRUD 사용 가능
- 동일 저장소 패턴: 리포지토리 메서드 이름 규칙도 Spring Data JPA와 동일

Refresh Token 엔티티 설계 기준

- 고유 식별자: 토큰마다 발급되는 고유 `jti` (JWT ID)를 엔티티 식별자로 사용
- 사용자 인덱싱: 사용자 단위 일괄 삭제를 위해 `username`에 `@Indexed` 부여
- 잔여 수명 필드: 남은 유효 기간을 초 단위 TTL 필드로 보관해 자동 만료 위임
- 최소 구성: 세션 판별에 필요한 필드만 두어 저장소 구조를 단순하게 유지

Refresh Token 엔티티 필드 선언

```
1  @RedisHash(value = "refreshToken")
2  public class RefreshToken {
3
4      @Id
5      private String jti;
6      @Indexed
7      private String username;
8      @TimeToLive
9      private Long ttl; // 초 단위 유효 기간
10 }
```

- 식별자·인덱스·TTL 세 필드만으로 세션 저장 구조를 완성.

엔티티 어노테이션의 역할

어노테이션	역할	결과
@RedisHash	클래스를 Redis 해시로 매핑	키 접두사(refreshToken)로 저장
@Id	엔티티 식별자 지정	jti 값이 Redis 키의 일부가 됨
@Indexed	보조 인덱스 생성	username 기준 검색 지원
@TimeToLive	만료 시간 필드 지정	값 초 경과 시 Redis가 자동 삭제

CrudRepository 기반 저장소 인터페이스

```
1 public interface RefreshTokenRepository
2     extends CrudRepository<RefreshToken, String> {
3
4     List<RefreshToken> findAllByUsername(String username);
5
6     void deleteAllByUsername(String username);
7 }
```

- 메서드 이름 규칙만으로 사용자별 조회·일괄 삭제 쿼리 자동 생성.

Redis 저장소 활성화 선언

```
1  @SpringBootApplication
2  @ConfigurationPropertiesScan
3  @EnableRedisRepositories
4  public class BoardApplication {
5
6      public static void main(String[] args) {
7          SpringApplication.run(BoardApplication.class, args);
8      }
9  }
```

- `@EnableRedisRepositories` 가 있어야 `@RedisHash` 리포지토리가 스캔됨.

Redis에 남는 키와 자동 만료



JwtProperties에 Refresh 수명 추가

```
1 @ConfigurationProperties(prefix = "app.jwt")
2 public record JwtProperties(
3     String secret,
4     Duration accessTokenValidity,
5     Duration refreshTokenValidity
6 ) {
7 }
```

- 하나의 프로퍼티 레코드로 두 토큰의 수명을 함께 관리.

핵심 용어 정리

인메모리 저장소 (In-Memory Store)

디스크가 아닌 메모리에 데이터를 두어 빠르게 읽고 쓰는 저장소

보조 인덱스 (Secondary Index)

기본 식별자 외의 필드로도 빠르게 검색할 수 있게 만든 색인

토큰 발급 로직 확장

토큰에 신원 외의 정보를 더 담기

- 고유 세션 키: `jti` 는 토큰마다 발급되는 고유 UUID이자 Redis 세션 키 역할
- 용도 구분 클레임: `type` 클레임으로 Access/Refresh 토큰의 쓰임을 구분
- 오용 차단 근거: 검증 시 `type` 값을 확인해 잘못된 용도의 토큰 사용을 차단
- 발급 시점 부여: 두 클레임 모두 토큰 생성 메서드에서 함께 채워 넣음

클레임 구성 비교

클레임	Access Token	Refresh Token
<code>sub</code>	사용자명	사용자명
<code>roles</code>	권한 목록 포함	미포함
<code>type</code>	<code>"access"</code>	<code>"refresh"</code>
<code>jti</code>	토큰별 고유 UUID	토큰별 고유 UUID(저장 키)

Access Token에 jti와 type 부여

```
1 return Jwts.builder()  
2     .id(UUID.randomUUID().toString())  
3     .subject(authentication.getName())  
4     .claim("roles", roles)  
5     .claim("type", "access")  
6     .expiration(Date.from(expiry))  
7     .signWith(key)  
8     .compact();
```

- 매 발급마다 새 **jti**를 부여해 토큰 단위 식별을 가능하게 함.

Refresh Token 발급 메서드

```
1 String jti = UUID.randomUUID().toString();
2
3 String token = Jwts.builder()
4     .id(jti)
5     .subject(username)
6     .claim("type", "refresh")
7     .expiration(Date.from(expiry))
8     .signWith(key)
9     .compact();
```

- 생성한 `jti` 를 변수로 별도 보관해 Redis 저장에 재사용.

발급 결과를 함께 전달하는 반환 타입

```
1 public record RefreshTokenIssue(  
2     String token, String jti, long ttlSeconds) {  
3 }  
4  
5 return new RefreshTokenIssue(  
6     token, jti, refreshTokenValidity.getSeconds());
```

- 토큰 문자열과 `jti`·TTL을 한 번에 반환해 Redis 저장 호출을 단순화.

type 클레임이 막아 주는 토큰 오용

오용 시나리오	검증 지점	차단 결과
Refresh를 Bearer 헤더로 API 호출	필터의 <code>type</code> 확인	<code>access</code> 가 아니므로 인증 거부
Access로 재발급 시도	재발급 로직의 <code>type</code> 확인	<code>refresh</code> 가 아니므로 거부

핵심 용어 정리

클레임 (Claim)

JWT 페이로드에 담기는 키-값 형태의 정보 조각

범용 고유 식별자 (UUID)

중복 없이 고유함을 보장하도록 생성되는 식별 값

쿠키 발급과 로그인 API 개편

Refresh Token을 응답 본문에 담지 않는 이유

- 탈취 경로 차단: JSON 본문에 담으면 JS가 읽을 수 있어 XSS 발생 시 그대로 탈취
- HttpOnly 속성: `document.cookie`로 접근 불가하여 스크립트 기반 탈취를 원천 차단
- 자동 첨부 범위: 브라우저가 지정된 인증 경로 요청에만 쿠키를 자동으로 첨부
- 분리의 이점: Access Token은 헤더로, Refresh Token은 쿠키로 전달 경로를 분리

쿠키 발급 책임을 전담하는 컴포넌트

```
1 public ResponseCookie createRefreshCookie(String refreshToken) {
2     return ResponseCookie.from(REFRESH_COOKIE_NAME, refreshToken)
3         .httpOnly(true)
4         .secure(secure)
5         .sameSite("Strict")
6         .path("/api/auth")
7         .maxAge(refreshTokenValidity)
8         .build();
9 }
```

- `AuthCookieFactory`가 쿠키 생성 로직을 한곳에 모아 일관된 보안 속성을 보장합니다.

로그아웃을 대비한 만료 쿠키

```
1 public ResponseCookie createExpiredRefreshCookie() {
2     return ResponseCookie.from(REFRESH_COOKIE_NAME, "")
3         .httpOnly(true)
4         .secure(secure)
5         .sameSite("Strict")
6         .path("/api/auth")
7         .maxAge(0)
8         .build();
9 }
```

- 빈 값과 `maxAge(0)` 을 지정해 브라우저에 저장된 쿠키를 즉시 삭제시킵니다.

쿠키 보안 속성의 선택

쿠키 속성	설정값	역할
<code>HttpOnly</code>	<code>true</code>	XSS를 통한 스크립트 탈취 방지
<code>Secure</code>	운영 <code>true</code> / 로컬 <code>false</code>	HTTPS 연결에서만 쿠키 전송
<code>SameSite</code>	<code>Strict</code>	크로스 사이트 요청 시 쿠키 전송 차단
<code>Path</code>	<code>/api/auth</code>	인증 경로로만 전송 범위 한정

로그인 응답의 이원화 설계

- Access Token 전달: JSON 응답 본문에 담아 클라이언트가 헤더 전송에 사용
- Refresh Token 전달: Set-Cookie 헤더로 HttpOnly 쿠키에 담아 전달
- 발급 이력 저장: 발급된 Refresh Token의 jti 를 Redis에 저장해 상태 관리
- 경로 분리의 이유: 두 토큰의 저장·전송 방식을 분리해 탈취 위험을 낮춤

로그인 시 두 토큰을 발급하고 저장

```
1 public LoginResult login(LoginRequest request) {
2     Authentication auth = authenticationManager.authenticate(
3         new UsernamePasswordAuthenticationToken(request.username(), request.password()));
4     String accessToken = jwtTokenProvider.createAccessToken(auth);
5     RefreshTokenIssue refresh = jwtTokenProvider.createRefreshToken(auth.getName());
6     refreshTokenRepository.save(
7         new RefreshToken(refresh.jti(), auth.getName(), refresh.ttlSeconds()));
8     return new LoginResult(new TokenResponse(accessToken), refresh.token());
9 }
```

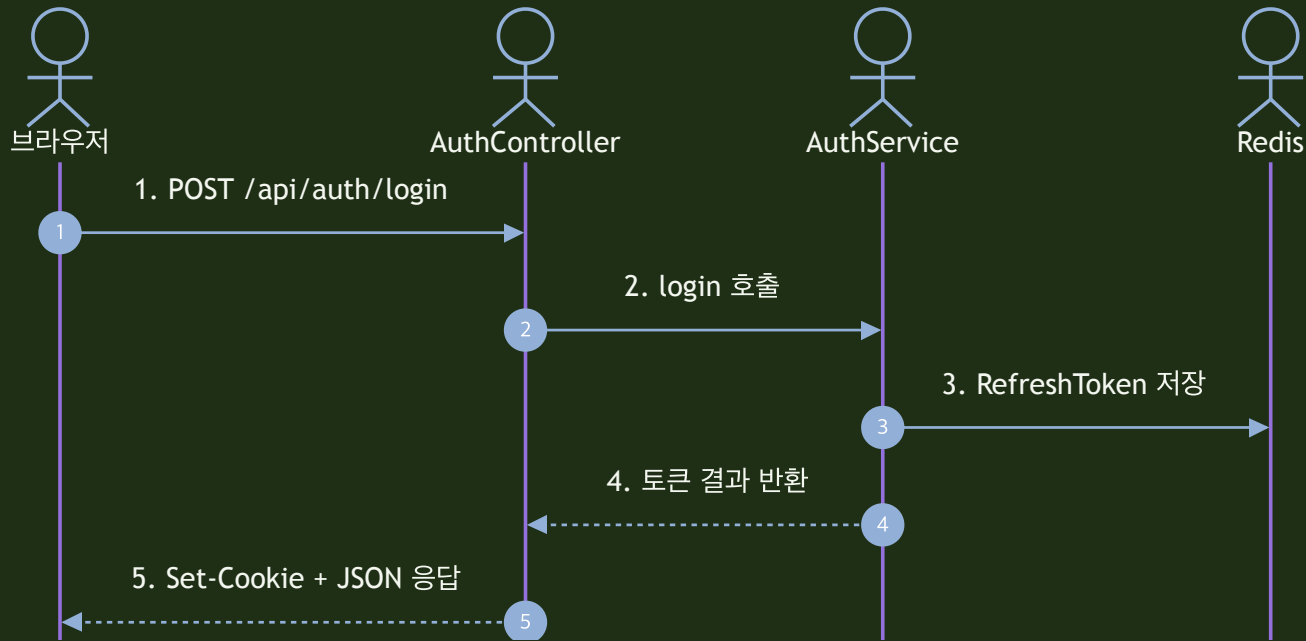
- 인증 성공 후 Access·Refresh Token을 함께 발급하고 발급 이력을 Redis에 저장합니다.

컨트롤러에서 쿠키와 본문 함께 응답

```
1  @PostMapping("/login")
2  public ResponseEntity<TokenResponse> login(@RequestBody LoginRequest request) {
3      AuthService.LoginResult result = authService.login(request);
4      ResponseCookie refreshCookie = authCookieFactory.createRefreshCookie(result.refreshToken())
5
6      return ResponseEntity.ok()
7          .header(HttpHeaders.SET_COOKIE, refreshCookie.toString())
8          .body(result.tokenResponse());
9  }
```

- **Set-Cookie** 헤더와 JSON 본문을 한 응답에 함께 실어 두 토큰을 동시에 전달합니다.

로그인 요청의 처리 흐름



핵심 용어 정리

HttpOnly 쿠키 (HttpOnly Cookie)

JavaScript의 `document.cookie` 접근을 차단하여 XSS로 인한 값 탈취를 막는 쿠키 속성

동일 사이트 정책 (SameSite)

요청이 발생한 사이트에 따라 쿠키 자동 전송 여부를 제어해 CSRF를 완화하는 쿠키 속성

학습 요약 (1/2)

- 무상태 JWT의 한계: 발급 후 로그아웃·탈취 상황에서 즉시 무효화가 불가능
- 이중 토큰 역할 분리: 짧은 Access Token과 긴 Refresh Token으로 보안·편의 동시 확보
- .env와 config.import: 민감 정보를 코드 밖에서 관리하고 `spring.config.import`로 주입
- Redis TTL 자동 만료: 발급 이력을 저장하고 만료 시점에 자동 삭제

학습 요약 (2/2)

- @RedisHash 저장소: 복잡한 설정 없이 객체 단위 CRUD와 TTL 관리 지원
- jti와 type 클레임: 토큰 식별과 오용 방지를 위한 클레임을 Access·Refresh에 부여
- HttpOnly 쿠키 속성: `Secure` · `SameSite` · `Path` 를 함께 지정해 다층 방어 구성
- 로그인 응답 이원화: 다음 차시는 무음 재발급·재사용 감지·블랙리스트 처리 예정

예상 질문과 답변

Q&A: 구현 (1)

- Q. Refresh Token도 Redis에 값 전체를 저장해야 하나요?
- A. 아니오. `jti`만 저장하고, 실제 토큰 검증은 서명 확인으로 처리하므로 값 전체 저장이 필요 없습니다.
- Q. `.env`를 안 만들면 애플리케이션이 뜨지 않나요?
- A. `optional:` 접두어를 붙였으므로 파일이 없어도 기동은 됩니다. 다만 프로퍼티가 해석되지 않아 Redis 연결 등에서 오류가 발생합니다.

Q&A: 구현 (2)

- Q. @TimeToLive 값을 안 넣으면 어떻게 되나요?
- A. TTL이 지정되지 않으면 데이터가 만료 없이 영구 보관되어 Redis에 계속 남게 됩니다.
- Q. 쿠키 Path를 /api/auth로 두면 재발급 외 요청에서 쿠키를 못 쓰는 것 아닌가요?
- A. 의도된 설계입니다. Refresh Token은 재발급·로그아웃 등 인증 경로에서만 필요하므로 그 외 노출을 막는 것이 목적입니다.

Q&A: 성능

- Q. 매 요청마다 Redis를 조회하면 느려지지 않나요?

- A. Access Token은 서명 검증만으로 처리되어 Redis를 조회하지 않으며, Redis 조회는 재발급 시점에만 발생합니다.

- Q. 만료된 토큰이 Redis에 쌓이지 않나요?

- A. @TimeToLive로 TTL이 지정되어 있어 만료 시 자동 삭제되며 별도의 배치 작업이 필요 없습니다.

Q&A: 실무

- Q. Access Token 수명은 실무에서 얼마로 잡나요?
- A. 보통 15~30분 사이로 설정하며, 서비스 민감도와 사용자 경험을 고려해 조정합니다.
- Q. 클라우드 Redis가 잠시 끊기면 로그인에 막히나요?
- A. 재발급 요청만 실패하고 이미 발급된 유효한 Access Token은 계속 동작합니다. timeout과 재시도 설정으로 영향을 최소화합니다.